Book Exchange Platform

Software Architecture and Design Specification

This document defines the architecture, relational data model, API contracts and security design for the Book Exchange Platform. A modular backend supports the short two-sprint delivery schedule while database transactions protect matching and exchange consistency.

Document control	Value
Version	1.0
Date	27 October 2026
Status	Submission baseline; implementation and test execution pending
Authors	Nishkal - DevOps and coordination; Abhiram - Auth; Manoj - Listings; Ravish - UI, documentation and demo.
Schedule	Sprint 1: 27-31 Oct; Sprint 2: 2-7 Nov; presentations from 9 Nov 2026
Revision	1.0 - Project baseline aligned to the updated 16-page plan and user-confirmed role swap.
Approval	Team and TA review pending; no signature or approval is claimed.

Contents

1 Introduction; 2 Document overview; 3 Architecture; 4 Detailed design; 5 Appendices. SRS requirement IDs and STP test IDs are the common traceability keys.

1 Introduction and 2 Document overview

1.1 Purpose and scope

Describe how the SRS is implemented. Cover authentication, listings, requests/matching, exchanges, ratings, admin, UI and deployment. Exclude payments, shipping and campus SSO. Intended readers are developers, testers and the course evaluator.

2.1 How to use this document

Read the component view before editing shared files. Use section 3.6 for migrations and section 4.3 for contracts. All endpoint examples have the /api/v1 prefix unless explicitly stated otherwise. Requirements are defined in SRS, tests in Test_Plan, and sprint work in the updated plan. Do not treat this design as evidence of implementation.

3 Architecture

3.1 Goals and constraints

Deliver authentication/listings in Sprint 1 and matching/exchanges in Sprint 2. Prioritize ownership checks, reversible schema migrations, consistent errors and repeatable deployment. Use one repository and a modular monolith to limit operational complexity. The course window is too short to justify independent microservices.

3.2 Stakeholders and concerns

Stakeholder	Concern and response
Student	Find books quickly; clear requests and private contact details.
Team	Stable shared contracts, one migration owner, independent PR review.
TA	Traceable requirements, meaningful CI, accurate Jira and demo evidence.
Operator	Health checks, secrets configuration, rollback and database durability.

3.3 Component view

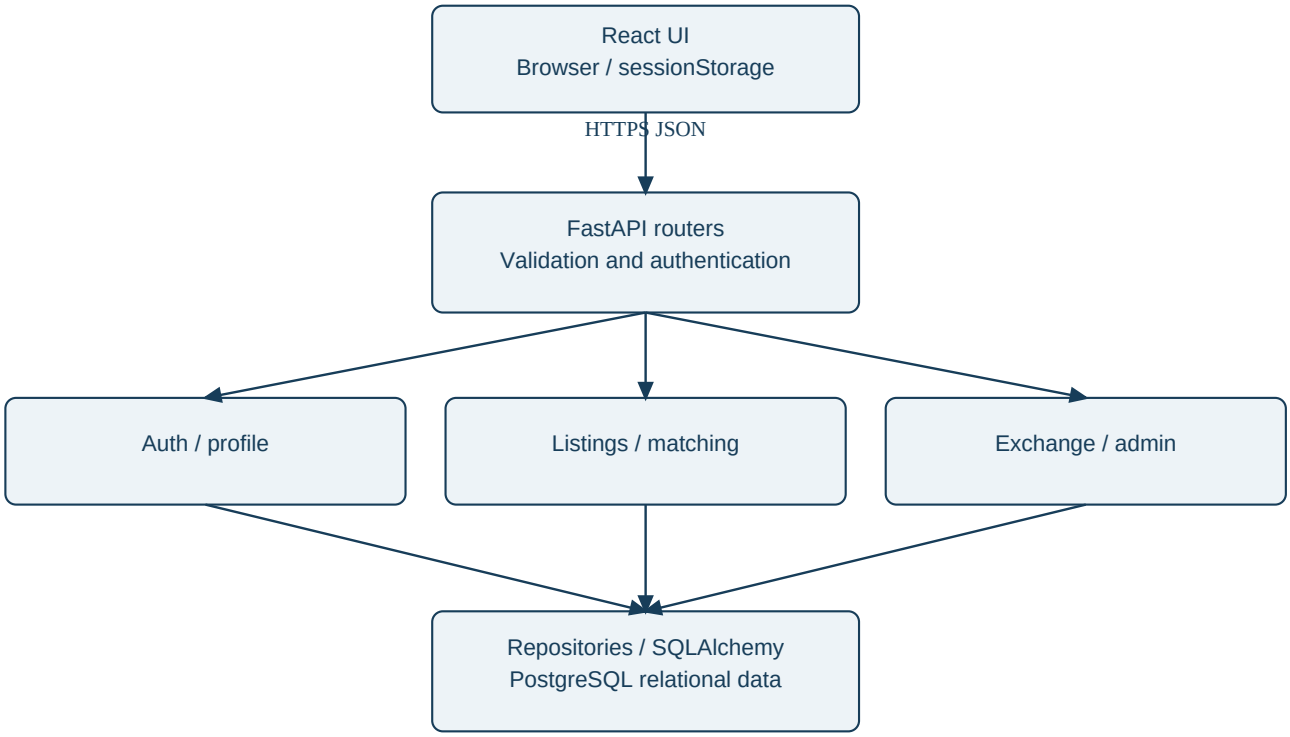

Figure 1. Layered component view. Arrows represent calls; PostgreSQL is accessible through the backend only.

3.4 Components and responsibilities

Component	Responsibility / requirements
React UI	Forms, navigation, loading/error handling and session state; NF-003.
Auth / profile	Hashing, JWT validation, login throttling, profile permissions; F-001..004, SR-001/002/006.
Listings	Validation, book reuse, listing ownership, search and soft delete; F-010..013, SR-004/005.
Requests and matching	Candidate selection, score calculation and acceptance locks; F-020..022/024.
Exchange	Handover, return, history, notifications and ratings; F-023/025/026/030/031.
Admin	Moderation, audit records and restricted contact projection; F-040, SR-007.
Repositories	Parameterized data access and explicit transaction boundaries.
DevOps	Migrations, builds, tests, deployment and probes; NF-002/004/006, SR-003.

3.5 Pattern and decisions

ADR-01: Use a layered modular monolith; routers validate HTTP, services enforce policy, repositories query data. Reject microservices for this release because distributed deployment and transactions add unnecessary risk. ADR-02: PostgreSQL provides constraints, locks and pg_trgm for search. ADR-03: JWT plus a database token_version supports immediate all-session logout; every protected request reads the active user. ADR-04: Keep a single-instance in-memory rate limiter for Sprint 1, recording its limits; use a shared store before scaling.

3.5.1 Matching algorithm

Normalize ISBN by removing hyphens; normalize textual search by trim/lowercase. Exclude deleted/unavailable listings, inactive owners and the borrower's own listings. For an ISBN request, require exact ISBN. For a text request, require title/author containment or title trigram similarity >=0.3. Score = 70*relevance + 20*same_location + 10*(condition_rank/4). Relevance is 1 for exact ISBN or title trigram similarity in [0,1] for text requests. Sort descending score, then newest listing, then listing ID for stable ties; return at most five. This scoring is a proposed Sprint 2 baseline, not a performance claim.

Suggestions do not reserve inventory. On acceptance, lock the request, listing and match in a consistent order; recheck eligibility and participant identity. Once both accept, create exactly one active exchange, mark listing RESERVED and request FULFILLED in one transaction. A partial unique index on active exchange listing_id and a unique active request constraint prevent concurrent winners. Re-match on decline, timeout or cancellation; retain history and mark obsolete suggestions stale.

3.6 Technology and data model

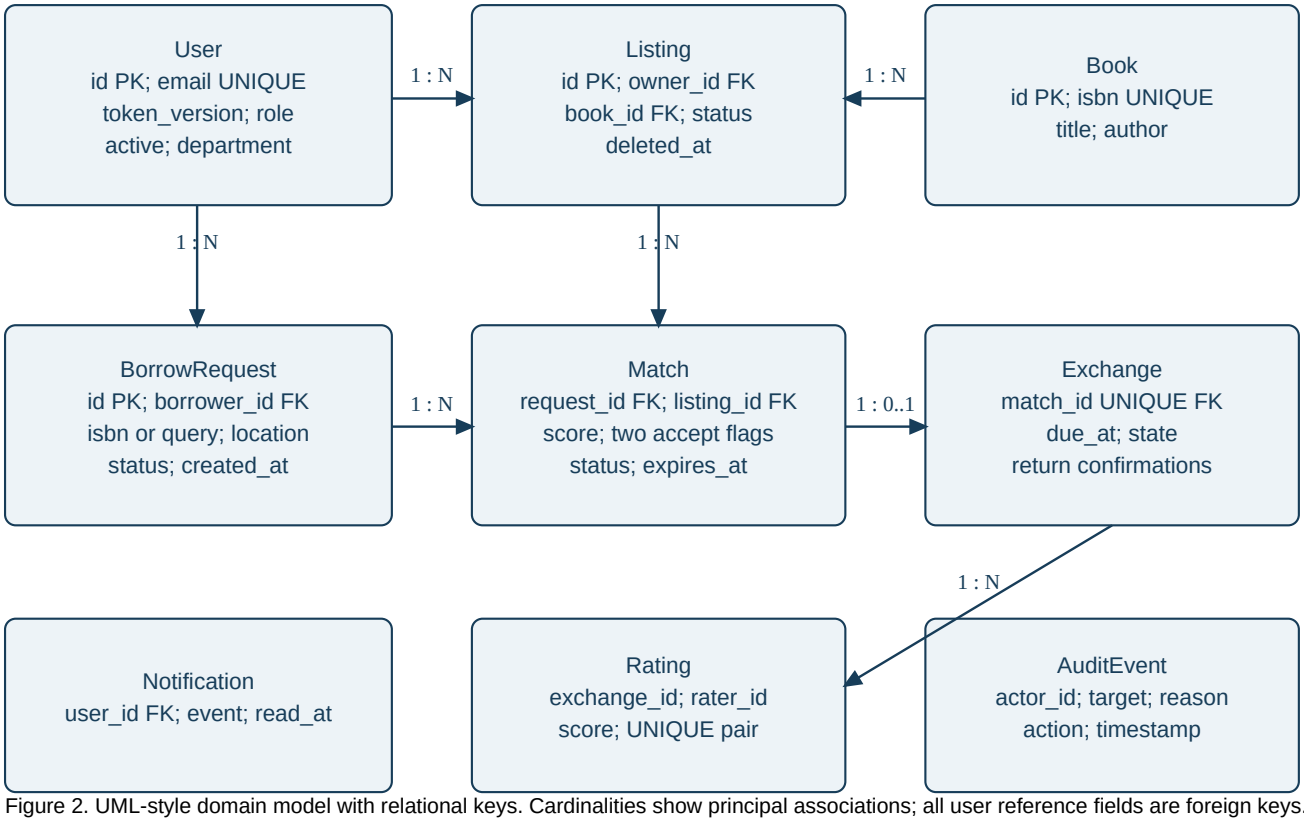

Figure 2. UML-style domain model with relational keys. Cardinalities show principal associations; all user reference fields are foreign keys.

Stack baseline: React/Vite and Axios; FastAPI/Pydantic; SQLAlchemy and Alembic; PostgreSQL 15 with pg_trgm; pytest, coverage and Ruff; GitHub Actions; Docker Compose. Python 3.11 and Node 20 follow the supplied plan. The scaffold will pin compatible dependencies and verify support and security before deployment.

Table	Additional constraints and fields
users	name, email unique lowercase, password_hash, role(student/admin), department nullable, location, avg_rating, active, token_version default 0.
books	ISBN unique; title and author lengths constrained. ISBN normalization happens before insert. Concurrent reuse relies on unique constraint and retry/read.
listings	owner_id/book_id NOT NULL FKs; condition_rank 1..4; status AVAILABLE/UNAVAILABLE/RESERVED/ON_LOAN; created_at and deleted_at UTC.
borrow_requests	borrower_id FK; ISBN or query required; OPEN/MATCHED/FULFILLED/CANCELLED; location and creation time.
matches	request_id/listing_id FKs; score; borrower/lender accepted flags; PROPOSED/DECLINED/EXPIRED/ACCEPTED; unique active request/listing pair.
exchanges	match_id unique; listing_id/request_id FKs; ACCEPTED/ON_LOAN/RETURNED/CANCELLED; due date, handover and return confirmations. Partial unique active listing and request indexes.
ratings / notifications / audit_events	Ratings unique(exchange_id,rater_id), score 1..5. Notifications unique(user_id,event_key), read_at nullable. Audit reason mandatory; writes restricted to service role.

3.6.1 Migrations and indexes

Sprint 1 first migration includes users, books and listings, token_version, department, deleted_at and CREATE EXTENSION IF NOT EXISTS pg_trgm. Add the ISBN unique index, GIN title gin_trgm_ops index and listing status index. Index owner_id and book_id for common joins. Inspect EXPLAIN ANALYZE for real search predicates; title similarity ordering may need sorting and author containment may still scan. Do not claim the index eliminates every scan.

Sprint 2 introduces requests, matches, exchanges, notifications, ratings and audit events with constraints before enabling those routes. Store timestamps in UTC. Back up the database before schema deployment; run a single migration job rather than multiple concurrent app startup migrations in production. Test fresh upgrade and rollback in a disposable environment.

3.7 Risks and mitigations

Risk	Mitigation
Duplicate reservations	Row locks, unique active constraints and concurrency tests.
Single-instance limiter resets	Document limitation; keep one worker for baseline; adopt shared storage before scaling.
Free hosting sleep or quotas	Record actual provider limits; warm demo; count outages in measured availability.
Shared model conflicts	DevOps owns initial models; feature PRs coordinate changes.
No university domain supplied	Keep configurable and require verified value before registration acceptance.
Session token exposure through XSS	Escape output, avoid unsafe HTML, restrictive CSP and short lifetime; consider HttpOnly cookies in a later revision.

3.8 Traceability

SRS section 8 maps all requirements to modules and tests. Auth owns F-001..004 and SR-001/002/006; Listings owns F-010..013 and SR-004/005; Requests/Matching owns F-020..022/024; Exchange owns F-023/025/026/030/031; Admin owns F-040/SR-007. DevOps owns transport, availability, CI and portability. Diagram flows cover F-001 and F-024.

3.9 Security architecture

Threat	Control
Spoofing	bcrypt hashes; signed expiring JWT; active-user and token_version checks.
Tampering	Server-side schemas, parameterized queries, ownership and transition checks.
Repudiation	Restricted audit records with actor, action, reason and timestamp.
Information disclosure	HTTPS; secret-free logs; public response schemas; contact only after mutual acceptance.
Denial of service	Login limit, bounded pagination and query validation; monitor latency and errors.
Privilege escalation	Ignore no privileged input: reject forbidden profile fields; server-side admin dependency.

Token claims: sub=user ID string, role, ver, iat and exp. Enforce HS256 explicitly and load the user for authorization rather than trusting a stale role claim. JWT_SECRET must be a strong secret supplied through environment configuration. Logout increments token_version atomically, invalidating all sessions. Passwords are 8+ characters with a letter/digit and at most 72 UTF-8 bytes for bcrypt. Never log Authorization headers, plaintext passwords, hashes or deploy hooks.

Client follows the plan's sessionStorage baseline. A 401 clears local auth; logout calls the server before clearing it. This storage is script-accessible, so avoid raw HTML rendering and keep dependencies reviewed. Contact data is returned by an exchange-only endpoint after participant and state checks. An administrator may see only data needed for moderation; that does not authorize bulk disclosure.

4 Detailed design

4.1 States and invariants

Listing: AVAILABLE -> RESERVED -> ON_LOAN -> AVAILABLE after confirmed return. Owners may toggle AVAILABLE/UNAVAILABLE before reservation. Soft deletion is permitted only when no active exchange exists. Match: PROPOSED -> ACCEPTED, DECLINED or EXPIRED. Exchange: ACCEPTED -> ON_LOAN -> RETURNED; ACCEPTED -> CANCELLED releases the listing. Invalid transitions return 409 without changing data. Contact is accessible only to participants of ACCEPTED/ON_LOAN/RETURNED exchanges, and never after cancellation.

4.2 Sequence diagram one

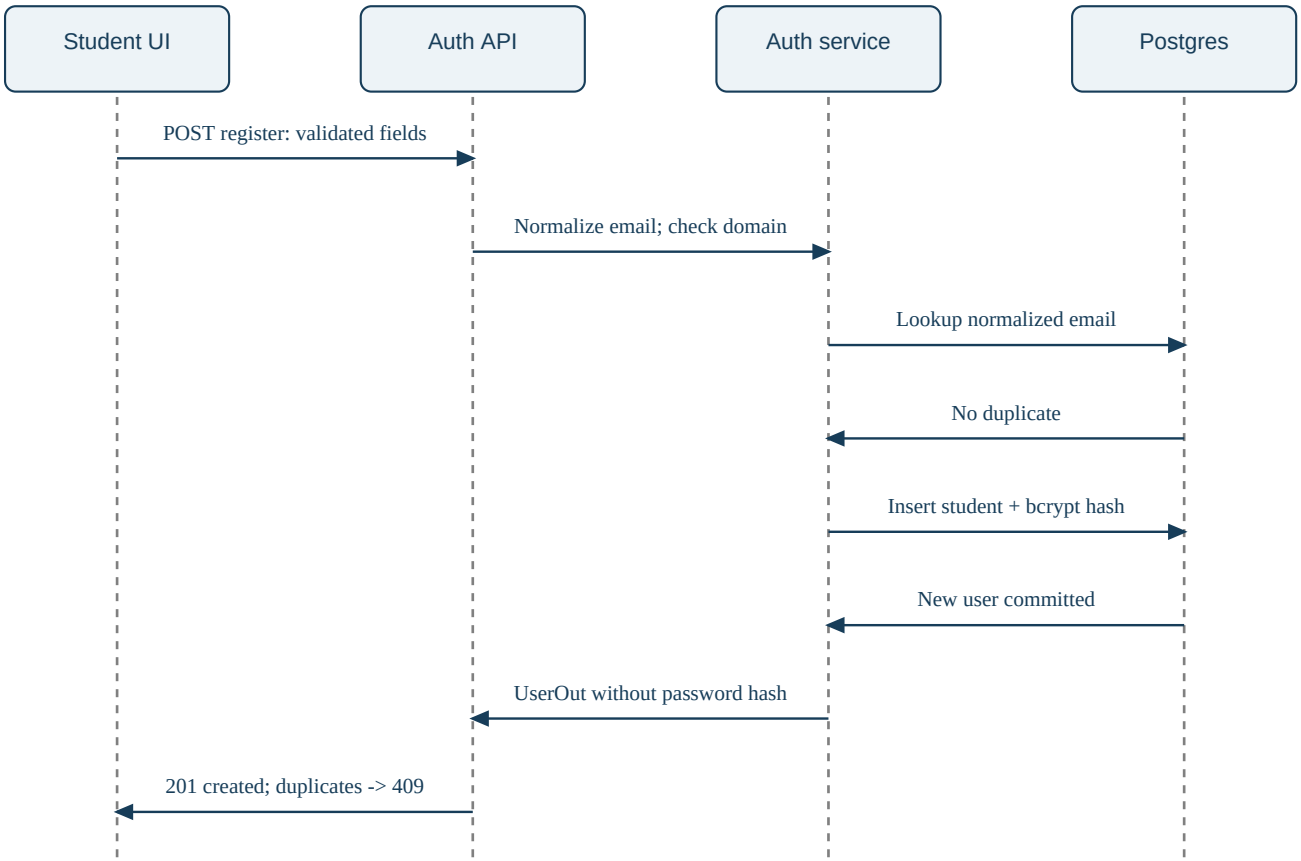

Figure 3. Registration success path with duplicate error branch. The database unique email constraint handles simultaneous registrations.

4.2 Sequence diagram two

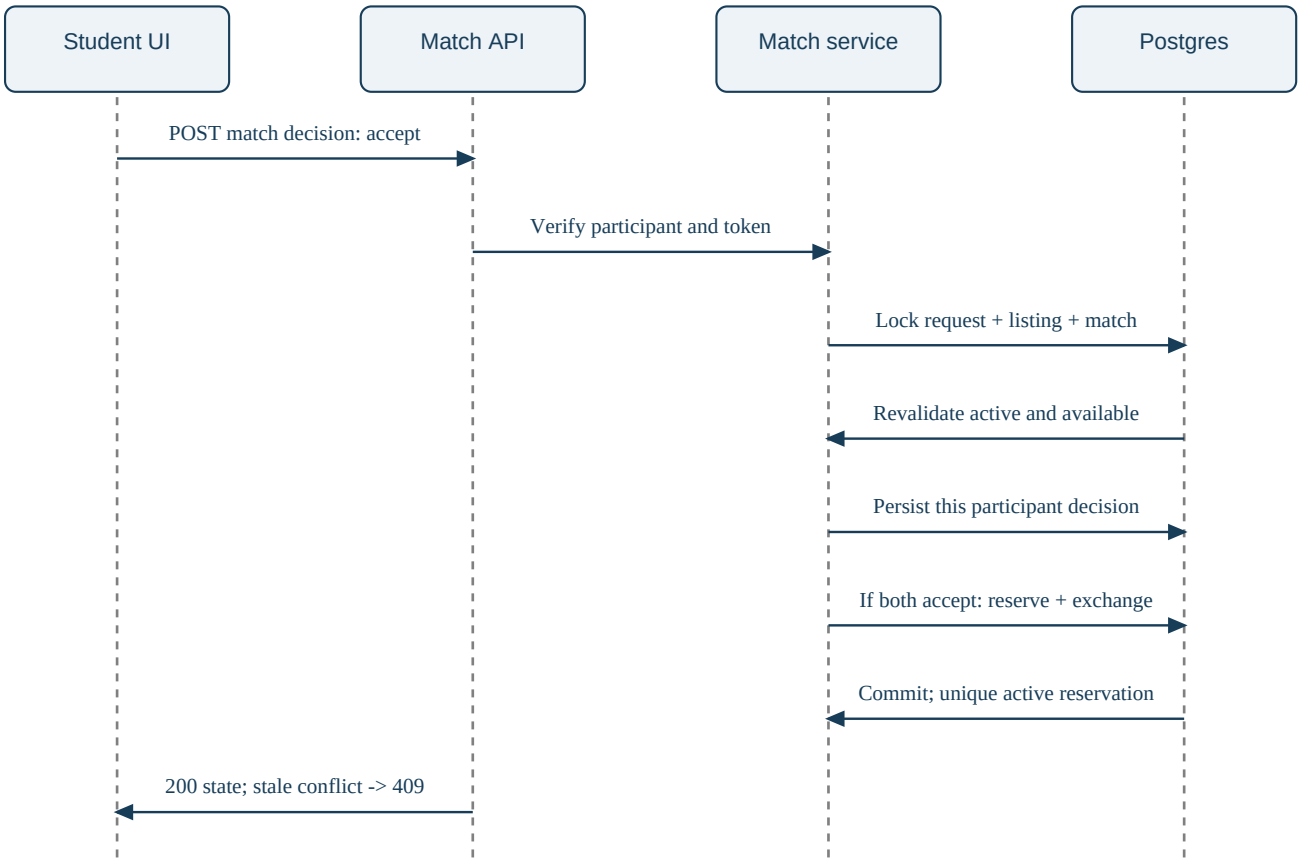

Figure 4. Participant acceptance with transaction locking. A second decision creates an exchange only if both participants accepted and the listing is still eligible.

For a single acceptance, commit only the decision and notify the other participant. On mutual acceptance, invalidate conflicting pending matches and notify their borrowers to request new suggestions. Persist notification events in the same transaction or an outbox so a crash cannot lose the state change notification.

4.3 API contracts

Base URL /api/v1. JSON request and response bodies. Authentication is required except registration, login and public listing search. Missing/invalid credentials ->401; valid user without permission ->403; absent/deleted resource ->404; conflict ->409; invalid schema ->422. Business errors use {error: {code, message}}; field errors add fields. All collection endpoints use page>=1,size<=100 and {items,page,size,total}.

Method and path	Input / success	Key errors
POST /auth/register	{name,email,password}; 201 UserOut. UserOut={id,name,email,role,department,location}.	400 INVALID_EMAIL_DOMAIN;409 EMAIL_EXISTS;422 validation
POST /auth/login	{email,password}; 200 {token,expiresIn:1800}.	401 INVALID_CREDENTIALS;429 RATE_LIMITED with Retry-After
POST /auth/logout	Bearer token; atomic token_version increment;204 no body.	401 invalid session
GET /users/me	200 UserOut for current user.	401
PUT /users/me	{name?,department?,location?};200 UserOut. Unknown/privileged fields rejected.	401;422
GET /listings	q,isbn,location,page,size;200 public ListingOut collection.	422
GET /listings/mine	Current user's non-deleted listings;200 paginated collection. Declare before /{id}.	401;422
POST /listings	{title,author,isbn,condition,location};201 ListingOut.	401;422
PUT /listings/{id}	Partial listing values/status;200 ListingOut. Changing shared book data uses get-or-create ISBN instead of modifying another owner's catalog entry.	401;403 NOT_OWNER;404;409;422
DELETE /listings/{id}	Soft delete;204 no body.	401;403 NOT_OWNER;404;409 ACTIVE_EXCHANGE

ListingOut={id,title,author,isbn,condition,location,status,ownerId,createdAt}; it omits contact details, password_hash and token_version. Existing ISBN metadata is reused rather than silently overwritten. An updated ISBN points to the appropriate book record; shared book edits are not allowed through listings.

4.3.1 Sprint 2 API contracts

Method and path	Contract	Errors
POST /requests	{isbn? or query?,location};201 {id,status:OPEN,...}.	401;409 duplicate;422
POST /requests/{id}/cancel	Owner;200 updated request; repeat cancel is idempotent.	403;404;409 active exchange
GET /requests/{id}/matches	Borrower only;200 top-five {matchId,listing,score,state}. No contacts.	403;404
POST /matches/{id}/decision	{decision:accept|decline};participant only;200 state and exchangeId if created.	403;404;409 stale
GET /notifications; PUT /notifications/{id}/read	Current user only;200 collection / updated notification.	401;404
PUT /exchanges/{id}/due-date	Either participant before handover;{dueAt};200 exchange. Partner sees agreed date before handover.	403;409;422
POST /exchanges/{id}/handover	Lender;future agreed due date required;200 ON_LOAN.	403;409
POST /exchanges/{id}/cancel	Participant before handover;200 CANCELLED.	403;409
POST /exchanges/{id}/return	Participant confirmation;200 state;both -> RETURNED.	403;409
GET /exchanges; GET /exchanges/{id}/contact	Participant history / authorized partner contact.	401;403;404
POST /exchanges/{id}/ratings	{score:1..5};201 rating after return.	403;409;422
POST /admin/users/{id}/deactivate; POST /admin/listings/{id}/hide	Admin and {reason};200 updated state plus audit event.	403;404;422
GET /admin/audit	Admin;paginated events, no credentials or tokens.	403

Sprint 2 contracts are proposed baselines for review on 2 November. Due-date agreement is recorded with one confirmation flag per participant; either date edit resets both flags. Handover requires both confirmations. Add POST /exchanges/{id}/confirm-due-date to set the caller's flag (200), rejecting past dates (409).

4.4 Errors, logging, deployment and monitoring

Use AppError(status,code,message) and a common validation handler. Logs contain request ID, route, status and duration; redact credentials and private fields. Roll back transactions on errors; uniqueness violations map to controlled conflicts, not raw SQL messages. /health returns {status:ok} only when the process and database check succeed; return 503 if the database is unavailable.

CI stages: backend lint/migration/pytest with >=70% coverage; frontend npm ci and production build; Docker build after both; deployment after Docker only for trusted main pushes. PR builds never deploy or receive production secrets. A temporary echo deploy job is a scaffold only; replace it with a real deployment and HTTPS smoke check. Branch rules require independent approval and the actual backend, frontend and docker checks once their first runs exist.

Deploy backend and persistent Postgres plus the static frontend on a chosen provider. Run migrations before rollout; verify /health and a smoke flow; roll back the app image on failure, and restore data only through a planned migration/backup procedure. Store deployment hooks and credentials in repository/hosting secrets. Configure a five-minute external uptime probe and retain the 2-7 November report.

4.5 UX design

Responsive navbar at 360px; labelled forms; 300ms search debounce; paginated cards; clear No books found state. My Listings uses the authenticated own-listing endpoint. Delete requires confirmation and shows 409 conflict messages. Profile edits only name, department and location. Do not expose internal infrastructure details in student-facing screens.

4.6 Open decisions

Confirm university domain, teammate account identities, hosting service and actual resource limits. Team review is needed for proposed matching score, availability and matching benchmark targets, and Sprint 2 ownership. Repository and Jira already exist; no ownership transfer or duplicate repository is required. The spreadsheet link has not been provided.

5 Appendices

Definitions: ADR = architecture decision record; ORM = object relational mapper; TLS = transport layer security; STRIDE = threat categories used in section 3.9. References: SRS v1.0, Test_Plan v1.0, supplied SAD template and updated 16-page task plan. Diagrams are maintained in docs/diagrams as SVG and PNG; this PDF embeds vector diagrams.

Planned deviations: token_version, department, deleted_at and /listings/mine are incorporated into this baseline. The Sprint 1 in-memory limiter and temporary active-exchange stub are explicitly limited and must be reviewed before final acceptance. No implementation results or security certification are claimed.